

シェルおよびスクリプト(主題1.06)

1. Linuxの標準シェル「bash」の主な環境設定ファイル

適用範囲	ファイル名	読込タイミング	主な設定内容
全ユーザー	/etc/bash.bashrc	bash起動時	Debian系) bash起動時に実行させたい処理(エイリアスなど)
	/etc/bashrc		RedHat系) bash起動時に実行させたい処理(エイリアスなど)
	/etc/profile	ログイン時	環境変数、利用環境にかかわるもの
各ユーザー個別	~/.bash_profile		環境変数などユーザー環境にかかわるもの
	~/.bash_login		~/.bash_profileがない場合の読み込み次候補 内容は~/.bash_profileと同じ
	~/.profile		~/.bash_loginがない場合の読み込み次候補 内容は~/.bash_profileと同じ
	~/.bashrc	bash起動時	bash起動時に実行させたい処理(エイリアスなど)
	~/.bash_logout	ログアウト時	ログアウト時に実行させたい処理

💡 知识点总结: Bash 配置文件的 “三国”

要看懂这张表，只需要问三个问题：

范围 (Scope): 是给全公司用的（`/etc/`），还是只给你自己用的（`~/`）？

时机 (Timing): 是推门进家（**ログイン時**），还是在屋里换衣服（**bash起動時**）？

内容 (Content): 是搬大件家具（**環境変数**），还是拿随身小件（**エイリアス**）？

🔍 选项解析与详细说明：文件全景图

全局配置 (全ユーザー用) —— `/etc/` 目录下

`/etc/profile`：

时机： 登录时。

内容： 设定 `PATH`、`LANG` 等全系统通用的环境变量。

`/etc/bashrc` (RedHat系) / `/etc/bash.bashrc` (Debian系)：

时机： 每次开启新终端时。

内容： 设定全系统通用的别名 (Alias) 或提示符 (PS1)。

个人配置 (各ユーザー用) —— `~/` 目录下

`~/.bash_profile`：

时机：登录时。

内容：你自己的私人环境变量。

`~/.bashrc`：

时机：每次开启新终端时。

内容：你最常用的 `alias`（比如 `alias ll='ls -l'`）。

`~/.bash_logout`：

时机：注销/退出时。

内容：清理临时文件、清屏等收尾工作。

 核心逻辑对标表 (考试秒杀)

文件类型	路径包含	针对对象	读取时机 (重要)	存放内容 (重要)
Profile系	/etc/profile ~/.bash_profile	全局 / 个人	ログイン時 (进门)	環境変数 (基础环境)
Bashrc系	/etc/bashrc ~/.bashrc	全局 / 个人	bash起動時 (开窗)	エイリアス (快捷缩写)
Logout	~/.bash_logout	个人	ログアウト時 (出门)	清理工作

 直接指向性口诀：五段连弹

- etc 管大家，波浪 (~) 管自家； (范围)
- Profile 迎宾门，Bashrc 随手礼； (时机)
- 环境进 Profile，习惯进 Bashrc； (内容)
- 首选 _profile，次选 _login，最后 .profile； (后补顺序)
- Logout 是注销，挥手告别活全消！ (退出)

 考试小秘籍

`source` 命令：修改完配置文件后，如果不重启，执行 `source ~/.bashrc`（或 `.` 命令）可以让配置立刻生效。

继承关系：虽然 `Profile` 只在登录时读一次，但通常它内部会有一行代码去“顺便”读取一下 `bashrc`，所以登录时两者都会生效。

非登录 Shell：如果你在图形界面里点开一个新的 Terminal 窗口，它**不会**读取 `profile`，只会读 `bashrc`。所以 Alias 一定要写在 `bashrc` 里。

2. Debian系のシステムで全ユーザーのbash起動時に共通の処理を実行させたい。編集すべきファイルは`/etc/bash.bashrc`
3. 環境変数のうち、コマンドやプログラムを検索するディレクトリのリストが格納されているものは**PATH**

PATH (路径) 搜命令，目录冒号 (:) 连成行；PWD (当前) 站原地，HOME (家) 才是避风港；USER (用户) 报大名，SHELL (程序) 解释最响亮！

 考试小秘籍

修改 PATH：如果你写了一个脚本想直接运行，需要把脚本所在的目录加入 `PATH`，通常在 `~/.bash_profile` 里写：`PATH=$PATH:/新路径`。

查看变量：在命令行查看这些变量的值，需要在名字前加 `$` 符号，例如：`echo $PATH`。

大写锁定：Linux 的标准环境变量名几乎全部是**大写**。

在 Linux 里，当你说“查找一个命令”时，系统其实是在做一桩“**按图索骥**”的体力活。

 知识点总结

用户的直觉：我要找命令的“名字列表”（List）。

系统的逻辑：我不知道命令在哪，你给我一个“**路径清单（PATH）**”，我挨个文件夹去搜。

 选项解析与详细说明：为什么是 PATH？

1. PATH 到底存了什么？

当你输入 `ls` 时，系统并不知道 `ls` 在哪里。它会打开 `PATH` 这个变量，发现里面写着：
`/usr/local/bin : /usr/bin : /bin`

这就是一个**目录的列表（List of Directories）**。

2. 系统的“找法”

去 `/usr/local/bin` 看看有没有叫 `ls` 的文件？（没有）

去 `/usr/bin` 看看有没有叫 `ls` 的文件？（没有）

去 `/bin` 看看有没有叫 `ls` 的文件？（找到了！运行它！）

所以，`PATH` 存储的是**“为了找到命令而需要搜寻的目录清单”**。

 考试小秘籍

关键词锁定：只要题目问「**コマンドを検索する**」（搜寻命令）或者「**ディレクトリのリスト**」（目录列表），答案雷打不动就是 `PATH`。

分隔符：记住路径之间是用**冒号** `:` 分开的，这也是常考的小细节。

优先级：写在 `PATH` 最左边的目录最先被搜到。如果你在两个地方装了不同版本的程序，左边的那个会生效。

4. Login Shell 启动时, Bash 只会直接读取一次文件 (按 `bash_profile` -> `bash_login` -> `.profile` 的顺序, 找到第一个就停)。剩下的文件全靠这些文件内部的 `source` 命令去“套娃”。

另外, 在 LPI 考试中, `~/.profile` 是一个非常关键的兼容性角色。`bash` 在登录时非常“挑剔”, 它会按顺序找这三个文件, 只要找到一个, 后面两个就不看了:

`~/.bash_profile` (亲儿子, 优先级最高)

`~/.bash_login` (二儿子, 极少用)

`~/.profile` (老祖宗/备胎, 优先级最低)

📊 登录 vs 非登录 加载路径表

模式	场景	主动读取的文件	间接读取的文件
Login Shell	SSH 登录 / 命令行登录	<code>~/.bash_profile</code>	→调用 <code>~/.bashrc</code>
Non-login Shell	图形界面点开终端	<code>~/.bashrc</code>	→调用 <code>/etc/bashrc</code>

5. 現在の環境変数PATHに「/home/user1/script」ディレクトリを追加したい。

空欄に当てはまるものはどれか。(2つ選択)

\$ export _____

`PATH = $PATH:/home/user1/script`

`PATH = /home/user1/script:$PATH`

6. Bash 的加载逻辑分为两类:

- a. 登录时 (ログイン時): 读取 `profile` 系列文件。
- b. 交互式启动 (起動時 / 非ログイン): 读取 `bashrc` 系列文件。

为什么选这两个?

当你在已经登录的状态下开启一个新的 `bash` 进程 (交互式非登录) 时:

- `~/.bashrc`: 这是个人定义的“随身礼”, 每开一个窗口都会读一次, 用来加载你的 `alias` (别名) 和个性化设置。
- `/etc/bashrc`: 这是全局定义的“随身礼”。通常情况下, `~/.bashrc` 的脚本内容里会包含一行命令, 主动去调用 (source) 这个 `/etc/bashrc`。

为什么其他选项不对?

- `/etc/profile` & `~/.bash_profile`: 这两位是“迎宾门”, 只有在**登录 (Login)**那一刻才会读取。你开新窗口时, 它们已经休息了。
- `~/.bash_logout`: 这是“送客门”, 只有在**退出 (Logout)**时才读取。
- `~/.bash_history`: 它根本不是配置文件, 它是记录你敲过哪些命令的**历史记录文件**。

7. エイリアスを削除するのではなく、一時的にエイリアスを無効にする場合は、エイリアスコマンドの前に「\」 バックスラッシュを入力してコマンドを実行します。
8. 这道题考查的是 **Bash 中“转义字符（エスケープ文字）”** 的一个巧妙用法。当你想要暂时跳过已经设置好的别名（Alias），直接调用系统最原始的命令时，这个小技巧非常实用。

💡 知识点总结

Alias (别名): 通常为了安全，Linux 会默认设置 `alias rm='rm -i'`，让你删除文件前必须确认。

反斜杠 (\): 在命令前加上反斜杠，可以**转义**掉别名。它的作用是告诉 Shell：“别看我的外号，请直接执行我的本尊（原始二进制文件）”。

1. \ (反斜杠) —— 正解

作用: 暂时屏蔽掉 Alias。

场景: 当你需要一口气删除几百个文件，不想被 `rm -i` 问几百遍“是否确认”时，可以使用 `\rm *`。

别名设置太啰嗦，反斜杠 (\) 一加变大哥；不问确认直接删，原始命令最利落！

📊 三种禁用别名的方法对比

方法	操作示例	特点
使用反斜杠 (最常用)	<code>\rm</code>	临时单次生效，最快。
使用绝对路径	<code>/bin/rm</code>	绕过所有 Alias，最稳。
使用引号	<code>"rm"</code> 或 <code>'rm'</code>	效果等同于反斜杠，较少用。
彻底取消	<code>unalias rm</code>	在当前会话中 永久 删掉别名。

📖 考试小秘籍

安全提醒: 在实际工作中，慎用 `\rm`。别名 `rm -i` 是为了防止误删，如果你用了反斜杠，文件删了就真的回不来了！

其他转义: 反斜杠不仅能取消别名，还能让特殊字符（如空格、`*`）失去特殊含义，变成普通的文字。

绝对路径: 如果考试问“除了 `\` 还有什么方法运行原始命令”，记得找 `/bin/rm` 这种带路径的选项。

9. 这道题考查的是 `~/.bash_logout` 的执行时机和功能。它是整个 Bash 配置文件加载生命周期的“最后一站”。

💡 知识点总结

执行时机: 它是在你退出登录（Logout / Exit）的**直前**（即将离开前）被 Bash 自动执行的。

主要功能: 用于执行一些“收尾工作”，比如清理临时文件、清屏、或者记录退出时间。

为什么其他选项不对？

「ログアウトの直後（之后）」：

错误：一旦退出，Shell 进程就结束了，没法再执行脚本。

「最終ログアウト情報が書き込まれている（记录时间的文件）」：

混淆：这说的是类似 `/var/log/lastlog` 或 `wtmp` 这种系统日志，而不是配置脚本。

「プロセスを停止する処理がデフォルトで記述（默认停止进程）」：

错误：这个文件默认内容通常非常少（或者干脆是空的，或只有一句 `clear`）。系统不会默认在这里写复杂的进程停止逻辑。

Profile 迎宾进大门，Logout 送客掩小门；直前（之前）动手搞卫生，清屏扫地 (Clear) 抹余痕！

 考试小秘籍

关键词锁定：看到「ログアウト（注销）」且选项里有「直前（之前）」，基本就是正确答案。

默认内容：在很多发行版中，这个文件默认只有一行 `clear`，目的是为了防止下一个人坐在你的电脑前看到你刚才在终端里输入的内容。

非登录 Shell 不执行：注意，如果你只是关掉一个普通的终端窗口（Non-login Shell），它通常不会读取这个文件。它只针对**登录 Shell**的退出。

10. unalias コマンドの書式は以下の通りです。

`unalias [-a] [エイリアス名]`

なお、オプション「-a」を指定すると、登録されている全てのエイリアスが削除されます。この場合、引数のエイリアスは無視されます。

11. bash のログイン時、環境設定ファイルを読み込む順番で「`/etc/profile`」2.

「`~/.bash_profile`」3. 「`~/.bashrc`」4. 「`/etc/bashrc`」

这道题考查的是 **Bash 登录时（ログイン時）** 配置文件的**绝对加载顺序**。这是一个经典的“套娃”过程，系统遵循“先公后私、层层嵌套”的原则。

- 💡 知识点总结
- 当你在控制台或通过 SSH **登录（Login）** 时，Bash 的读取逻辑如下：
- 全局第一：**首先读取 `/etc/profile`（对所有用户生效）。
- 个人第二：**接着寻找个人配置文件（只找第一个存在的）：`~/.bash_profile`
→ `~/.bash_login` → `~/.profile`。
- **嵌套调用：**在 `~/.bash_profile` 内部，通常会有一行代码去读取 `~/.bashrc`，而 `~/.bashrc` 内部又会去读取 `/etc/bashrc`。

为什么是这个顺序？（核心逻辑）

第1步（`/etc/profile`）：系统大总管先出来，设定全员统一的环境变量（如语言、全局 PATH）。

第2步（`~/.bash_profile`）：轮到你自己的私人管家。**注意：这是系统最后一次“主动”读取文件。**

第3步 (`~/.bashrc`): 这是由 `~/.bash_profile` 内部代码手动调用的 (`~/.bashrc`) 。

第4步 (`/etc/bashrc`): 这是由 `~/.bashrc` 内部代码再次手动调用的。

12. ログインシェルの起動時に実行されるファイルと無関係なものは

`~/.bash_logout`

`~/.bash_history`

13. シェル変数の特殊変数

変数	説明
<code> \$? </code>	直前に実行されたコマンドの終了ステータス (exit値)
<code> \$n </code>	シェルスクリプトに渡されたn番目の引数。 (10個目以降は <code> \${10} </code> のように指定する。 「 <code> \$0 </code> 」はシェルスクリプト名)
<code> \$* </code>	全ての引数。 ダブルクォーテーションで「 <code> \$* </code> 」とした場合、 全ての引数がまとめられて1つの文字列となる
<code> \$@ </code>	全ての引数。 ダブルクォーテーションで「 <code> \$@ </code> 」とした場合、 それぞれの引数が個々の文字列となる
<code> \$\$ </code>	現在のシェルのPID (プロセスID)
<code> \$# </code>	実行時に指定された引数の数

这张表里的内容叫「特殊变数」。它们像是一组**“快捷状态灯”**，让你不需要写复杂的代码，就能立刻知道刚才发生了什么、现在是谁在运行。

💡 知识点总结

特殊变量是不需要你定义的，Bash 系统自动填充内容。它们主要负责三件事：

查结果：刚才的命令成功了吗？ (`$?`)

拿参数：脚本执行时后面跟了什么词？ (`$n` , `$*` , `$@` , `$#`)

查身份：现在的进程号是多少？ (`$$`)

🔍 选项解析与详细说明（深度记忆）

1. `$?` (終了ステータス)

含义：上一条命令的“成绩单”。

数值：0 代表满分（成功），非0 代表挂科（错误）。

记忆法：“刚才结果怎么样 (?)” —— 问号就是提问结果。

2. \$n (引数)

含义：第几个参数。\$0 是脚本自己的名字。

规则：\$1 是第一个，\$2 是第二个。超过 10 个要加花括号 \${10}。

记忆法：“Number (n)” —— 对应数字位置。

3. \$# (引数の数)

含义：一共有多少个参数。

记忆法：“# 是数量符号” —— 就像社交媒体上的话题标签，数数有多少个。

4. \$* 与 @\$ (全ての引数)

区别：

\$*：把所有参数打包成一个大包裹 ("1 2 3")。

@\$：把参数分成一个个独立的小盒子 ("1" "2" "3")。

记忆法：

* (星号)：像一个黑洞，把所有东西吸在一起混为一谈。

@ (At符)：像一个邮箱地址，每个人都有自己独立的地址（个体）。

5. \$\$ (プロセスID / PID)

含义：当前 Shell 自己的进程身份证号。

记忆法：“钱 (\$ \$) 很有权势” —— 有了 PID，系统才能管理这个进程。或者看形状，像两个站岗的卫兵。

📊 特殊变量秒杀记忆表

变量	核心含义	脑内联想
\$?	执行结果	“行不行？” (问号提问)
\$0	脚本名	“从零开始” (脚本的名字是源头)
\$#	参数个数	“Number 符号 #” (数个数)
\$\$	进程 PID	“身份证 \$\$” (双重加密的 ID)
\$*	所有参数(整体)	“星号大杂烩” (合体)
@\$	所有参数(个体)	“@ 每个人” (各归各)

📖 考试小秘籍

高频考点：\$? 绝对是考得最多的。记得 0 为真 (成功)，这和普通编程语言逻辑相反！

细节考点：\$* 和 @\$ 的区别。记住加了双引号之后，"\$@" 才是保留参数原貌的最佳实践。

写法规范：超过 9 的参数必须用 \${10} 这种形式。

脚本名字是 `$0` ，（领队 0 号）

参数个数看 `$#` ，（井号数格，数数有几个）

全部参数 `$*` 艾特 (`$@`)，（星星艾特，全体集合）

最后结果问问号 (`$?`)，（问号调查，成败代价）

左移一位用 `shift` 。（保安喊话，第一个滚蛋）

14. シェルスクリプトに渡された引数の数が2つではない場合は、後続の処理を中断するようなシェルスクリプトを作成したい。下图の()内に記述する内容として適切なものは次のうちどれか。

```
#!/bin/bash
if [ ( ) != 2 ]
then
    echo "引数の数が2つではなく( )のため、処理を中断します。"
    exit 1
fi
echo "指定された引数は $1 と $2 です。"
```

💡 知识点总结

- `$#` ：代表传递给脚本的**参数个数**（引数の数）。
- `$*` ：代表所有的参数列表。
- `$?` ：代表上一个命令的退出状态（終了ステータス）。
- `$` ：代表当前脚本的进程 ID (PID)。
- `$0` ：代表脚本自身的名称。

✅ 正确答案

`$#`

🔍 选项解析与详细说明

我们来看脚本的具体逻辑：

a. 需求分析：

- 题目要求：当“引数的数（参数个数）” **不是 2** 的时候，中断处理。
- 脚本中使用了 `if [ ( ) != 2 ]` 来进行判断。

b. 变量选择：

- 为了获取当前脚本接收到了几个参数，必须使用 `$#` 。
- 代入后变为 `if [ $# != 2 ]` ，即：如果参数个数不等于 2，则进入 `then` 分支执行退出操作。

🧠 小学生级记忆法：报数员

在这个脚本里，括号 () 里缺一个“报数员”。# 号长得就像个梯子，我们要顺着梯子数一数到底进来了几个人（参数）。只要看到题目问“数（かず）”或者“个数”，就大声喊出带“#”号的那个！

考试小秘籍

c. 关键词匹配：题目中出现「引数の数」，直接在选项里找 \$# 。

d. 区分含义：

- 如果你想在报错信息里显示脚本的名字，用 \$0 。
- 如果你想知道参数的具体内容，用 \$1，\$2 或 \$* 。
- 算总数，永远用 \$# 。

15. シェル変数の特殊変数

変数	説明
\$?	直前に実行されたコマンドの終了ステータス (exit値)
\$n	シェルスクリプトに渡されたn番目の引数。 (10個目以降は\${10}のように指定する。 「\$0」はシェルスクリプト名)
\$*	全ての引数。 ダブルクォーテーションで"\$*"とした場合、 全ての引数がまとめられて1つの文字列となる
\$@	全ての引数。 ダブルクォーテーションで"\$@"とした場合、 それぞれの引数が個々の文字列となる
\$\$	現在のシェルのPID(プロセスID)
\$#	実行時に指定された引数の数

这张表里的内容叫「特殊变数」。它们像是一组**“快捷状态灯”**，让你不需要写复杂的代码，就能立刻知道刚才发生了什么、现在是谁在运行。

我们用最直观的方式把这些符号和它们代表的意义“挂钩”：

\$? (終了ステータス)

- 含义：上一条命令的“成绩单”。
- 数值：0 代表满分（成功），非0 代表挂科（错误）。
- 记忆法：“刚才结果怎么样(?)”——问号就是提问结果。

\$n (引数)

- 含义：第几个参数。\$0 是脚本自己的名字。
- 规则：\$1 是第一个，\$2 是第二个。超过 10 个要加花括号 \${10}。

- 记忆法：“Number (n)” —— 对应数字位置。

\$# (引数の数)

- 含义：一共有多少个参数。
- 记忆法：“# 是数量符号” —— 就像社交媒体上的话题标签，数数有多少个。

*与@ (全ての引数)

- 区别：
- \$*：把所有参数打包成一个大包裹 ("1 2 3") 。
- \$@：把参数分成一个个独立的小盒子 ("1" "2" "3") 。
- 记忆法：
- *(星号)：像一个黑洞，把所有东西吸在一起混为一谈。
- @ (At符)：像一个邮箱地址，每个人都有自己独立的地址（个体）。

\$\$ (プロセスID / PID)

- 含义：当前 Shell 自己的进程身份证号。
- 记忆法：“钱 (\$ \$) 很有权势” —— 有了 PID，系统才能管理这个进程。或者看形状，像两个站岗的卫兵。

问号 \$? 问结果，零 (0) 成功才稳妥；

井号 \$# 数个数，几个兄弟排排坐；

数字 \$n 领位子，老大 \$1 最先过；

零号 \$0 是脚本，自报家门不啰嗦；

钱号 \$\$ 身份证，进程 PID 躲不过；

星号 \$* 大杂烩，艾特 \$@ 挨个落！

考试小秘籍

1. **高频陷阱**：考试最喜欢考 \$?。记住，如果你运行一个错误的命令（比如 ls /不存在的目录），立刻执行 echo \$?，结果一定是**大于 0** 的数字。
2. **脚本名称**：记住 \$0 永远是脚本自己，所以参数是从 \$1 开始算的。
3. **双引号威力**："\$*" 和 "\$@" 的区别只有在加了**双引号**时才最明显，如果不加引号，两者表现差不多。

16. 以下の図は、カレントディレクトリでのlsコマンドの実行結果と、test.shの内容である。

実行結果について、正しいものを三つ選べ。

```

$ls -l
合計 12
drwxrwxr-x. 2 user1 user1 6 1月 13 16:04 dir1
drwxrwxr-x. 2 user1 user1 6 1月 13 16:04 dir2
-rwxr--r--. 1 user1 user1 59 1月 13 16:11 test.sh
-rw-rw-r--. 1 user1 user1 2 1月 13 16:03 text1
-rw-rw-r--. 1 user1 user1 2 1月 13 16:03 text2

$cat test.sh
if test (A) $1
then
  echo "有効"
else
  echo "無効"
fi

```

カレントディレクトリの内容

スクリプトの内容

スクリプトの引数が「dir1」、(A)が「-e」の場合、「有効」と表示される

スクリプトの引数が「text3」、(A)が「-e」の場合、「無効」と表示される

スクリプトの引数が「text1」、(A)が「-e」の場合、「無効」と表示される

スクリプトの引数が「text1」、(A)が「-d」の場合、「無効」と表示される

スクリプトの引数が「dir1」、(A)が「-f」の場合、「有効」と表示される

スクリプトの引数が「dir3」、(A)が「-f」の場合、「有効」と表示される

这道题结合了我们刚学过的**特殊变量 \$1** 和 Linux 中非常高频的 **test 命令（文件测试）**。解题的关键在于：**看懂参数 \$1 到底指代哪个文件，以及测试开关（-e, -d, -f）到底在查什么。**

💡 知识点总结

- **\$1**：脚本执行时跟在后面的第一个参数（文件名）。
- **test 常用开关（常考点）**：
 - **-e (exists)**：只要文件或目录**存在**，结果就是真。
 - **-d (directory)**：必须是**目录**，结果才是真。
 - **-f (file)**：必须是**普通文件**（不是目录），结果才是真。

🔍 选项解析与详细说明（手把手排查）

我们对照 ls -l 的结果图来逐一确认：

引数 dir1 + -e (存在判断)

- **分析**：图中确实有 dir1 这个目录。-e 只要存在就通过。
- **结果**：test -e dir1 为真 \rightarrow 输出 **有効**。（正确 ✅）

引数 text3 + -e (存在判断)

- **分析**：图中只有 text1 和 text2，**没有 text3**。

- 结果：test -e text3 为假 \rightarrow 输出 無効。(正确 ✓)

引数 text1 + -d (目录判断)

- 分析：图中 text1 的权限开头是 - (-rw-rw-r--)，说明它是文件而不是目录（目录开头是 d）。
- 结果：test -d text1 为假 \rightarrow 输出 無効。(正确 ✓)

✗ 为什么其他选项不对？

- 引数 text1 + -e：text1 存在，所以应该显示“有效”。(错误)
- 引数 dir1 + -f：dir1 是目录，不是普通文件。-f 判断会失败，应显示“无效”。(错误)
- 引数 dir3 + -f：图中根本没有 dir3，必然显示“无效”。(错误)

-e (Exist) 查存在，只要活着就通过；

-d (Dir) 查目录，开头带 d 才算数；

-f (File) 查文件，普通文件才算对！

📖 考试小秘籍

1. 先看 ls -l 的第一位：

- 看到 drwxrwxr-x \rightarrow 它是目录（-d 才能过）。
- 看到 -rwxr--r-- \rightarrow 它是文件（-f 才能过）。

2. 分清 \$1：在本题中，\$1 就是你测试的对象。如果传入 dir1，脚本里执行的就是 test -e dir1。

3. 注意逻辑反转：有时候题目会考 ! test，那是取反的意思，要格外小心。

17. -r、-w 和 -x。

在 Linux 权限体系中，这三个字母分别代表了 **Read**（读）、**Write**（写）、**Execute**（执行）。

💡 知识点总结

- -r (Readable)：测试文件是否可读。
- -w (Writable)：测试文件是否可写。
- -x (Executable)：测试文件是否可执行（对于目录来说，代表是否可以进入该目录）。

✓ 正确答案

1. 「-r」はパスが存在し、かつそのパスが読み取り可能であれば真となる
2. 「-w」はパスが存在し、かつそのパスへの書き込みが可能であれば真となる
3. 「-x」はパスが存在し、かつそのパスの実行権限があれば真となる

为什么这三项是正确的？

这三个表达式的逻辑非常严谨，分为两步判定：

- **第一步**：文件/路径必须**存在**。
- **第二步**：当前用户必须拥有对应的**权限**。

只有这两个条件同时满足，test 命令才会返回 **0**（真）。

✗ 为什么其他选项不对？

- 「-rwx というように組み合わせられる」：
- **错误**：在 test 命令或 [] 表达式中，这些开关**不能**合并写成 -rwx。你必须分开写成 [-r file] && [-w file] && [-x file]。
- 「ファイルサイズが 0 であった場合は偽となる」：
- **错误**：这三个开关只管“权限”，不管“内容”。即使文件是空的（0 字节），只要你有权限读写，它依然返回真。
- **补充**：检查文件大小是否大于 0，使用的是 -s（Size）开关。

考试小秘籍

1. 权限 vs 属性：

- -e, -f, -d 查的是“**你是谁**”（存在、文件、目录）。
- -r, -w, -x 查的是“**你能干什么**”（读、写、执行）。

2. **注意用户身份**：这些测试结果是基于**当前执行脚本的用户**的权限。如果你是 root，基本上全是真；如果你是普通用户，就要看 ls -l 显示的权限位了。

3. **目录的 -x**：这是一个常考点。如果一个**目录**没有 -x 权限，你甚至无法用 cd 进入它。

18. 这道题考查的是 Shell 脚本中最重要的第一行：**Shebang（シバン）**。

知识点总结

- **Shebang**：指的是脚本开头由 #! 组成的字符序列。
- **作用**：告诉内核（Kernel）应该调用哪一个**解释器（Interpreter）**来运行这个脚本。如果第一行写错了，系统可能不知道该用什么语言去读它。

書式：#!インタープリタ

为什么是 #!?

- #：在 Shell 中代表注释。
- !：这是一个特殊标记。
- 合在一起，#! 就像是一个特殊的“启动信号”，系统看到它后会忽略后面的注释功能，转而加载指定的路径（如 /bin/bash）。

#!/bin/bash：加了一个点 . 代表相对路径，这是非常危险且不规范的。Shebang 必须使用**绝对路径**。

#! 就像是脚本的“报警电话”：

是拨号键，!是感叹号（紧急情况）；

后面跟着地址 /bin/bash，告诉系统去哪找“bash 警察”来处理这段代码！

考试小秘籍

1. 绝对路径：考试常考 /bin/bash 这种以 / 开头的绝对路径。
 2. 第一行限制：Shebang 必须出现在文件的第一行。如果你在它前面加了一个空行或空格，它就会失效，变成普通的注释。
 3. 权限配合：写好了 #!/bin/bash 后，通常还需要给文件增加执行权限（chmod +x script.sh），否则依然无法直接运行。
19. 这道题考查的是 Bash 中的**数值比较运算符**。在 Shell 脚本里，处理数字的大小时，我们不使用常见的 < 或 >（因为它们在 Shell 里被用来做“重定向”了），而是使用由两个字母组成的缩写。

知识点总结

- **数值比较**：必须使用带有连字符 - 的缩写。
- **核心逻辑**：
- **\$NUM1 (9) 和 \$NUM2 (34) 进行比较**。
- 题目要求：当 **9 小于 34** 时，输出 [OK]。
- **英语关键词**：Less Than（小于）。

正确答案

-lt

（完整语句为：if test \$NUM1 -lt \$NUM2）

选项解析与详细说明

a. -lt (Less Than) —— 正解

- **含义**：小于。
- **判定**：9 小于 34 成立，执行 then 后面的语句。

为什么其他选项不对？

- **-gt (Greater Than)**：大于。9 不大于 34。
- **-ge (Greater or Equal)**：大于或等于。
- **-le (Less or Equal)**：小于或等于。虽然 9 也满足“小于或等于”34，但题目通常要求最精准的“小于”关系。
- **-eq (Equal)**：等于。
- **-ne (Not Equal)**：不等于。

小 L (**L** ittle) 就是 **-lt** (小于)；

大 G (G reat) 就是 -gt (大于);

加个 e (E qual) 就是“带等于”:

-le (小于等于), -ge (大于等于)。

(注意: 数字比较必须带杠 -, 不能用括号!)

20. trap コマンドは、シェルに割り込みなどのシグナルが送られた時の処理を設定・実行します。

書式: trap [コマンド] シグナル

这道题考查的是 Bash 脚本中的信号处理（シグナル処理）。当你在脚本运行过程中按下 Ctrl+C（中断信号）或者系统要求脚本停止时，你可以预先设定好一段“遗言”或清理工作，这个动作就是由 trap 命令负责的。

💡 知识点总结

- 信号 (Signal): 是 Linux 系统用来通知进程发生了某件事（如：中断、挂起、终止）。
- trap 命令: 它的作用是“捕捉”这些信号，并执行你指定的命令。
- 常见用途: 脚本退出前自动删除临时文件、忽略用户的 Ctrl+C 干扰、记录日志等。

这几个概念其实是脚本里的**“应急预案”。我们把脚本想象成一个正在跑马拉松的运动员**，这些“信号”就是比赛中途发生的各种突发状况。

💡 知识点总结

- 信号 (Signal): 是外部给脚本发的“指令”（比如：喂，快停下！）。
- trap: 是脚本预设的“条件反射”，专门用来应对这些指令。

🔍 选项解析与详细说明

1. 捕捉 EXIT (0) —— “临终遗言”

- 原文意思: 不管脚本是怎么死的，死前必须执行某件事。
- 大白话:

假设你的脚本在运行时生成了一个临时文件 tmp_file。

- 如果脚本正常跑完了，它会删掉这个文件。
- 如果脚本跑到一半报错了，它也会删掉这个文件。
- 甚至你手动把它关掉了，它还是会执行删除。
- 生活例子: 就像你出门前设定一个规则：“无论如何，只要我离开家，必须关灯。”这样你不管是正常出门、还是失火逃命、还是被外星人抓走，灯都会关掉。

2. 忽略信号 (trap "" 2) —— “装聋作哑”

- 原文意思: 用户按 Ctrl+C 想打断我，我不理他。
- 大白话:

通常你在终端按 Ctrl+C（信号编号是 2），正在运行的程序会立刻停止。

但如果你在脚本开头写了 `trap "" 2`（中间是空的，代表“啥也不干”），那么：

- 用户疯狂按 Ctrl+C...
- 脚本：“我听不见，我继续跑。”
- 生活例子：就像你戴着降噪耳机写作业，哪怕旁边有人大喊大叫，你也完全不理睬，继续写。

3. 无法捕捉的 9 (SIGKILL) —— “强制执行”

- 原文意思：kill -9 是最高权力，谁也拦不住。
- 大白话：

虽然 trap 可以捕捉很多信号（比如 Ctrl+C），但 Linux 里有一个终极杀招叫 9 号信号（kill -9）。

- 这个信号是发给操作系统的，不是发给脚本的。
- 操作系统接收到 9，会直接把脚本的“电源”拔掉。
- 脚本没有任何机会执行 trap 里的命令。
- 生活例子：trap 就像是运动员的心理素质，能抗住压力和干扰。但 9 号信号就像是一道闪电直接劈下来，运动员连留遗言的时间都没有，直接物理消失。

21. 作成したシェルスクリプトが意図通りの動作をしないが、要因が思い当たらないため当該スクリプト全体の処理の流れを追って確認したい。

这道题考查的是 **Bash 脚本的调试（デバッグ）手段**。当你不知道脚本哪里写错时，最有效的办法就是让脚本在运行的时候把“每一步干了什么”都打印出来，这就是所谓的“**执行追踪（実行トレース）**”。

💡 知识点总结

- **-x 选项**：这是 Bash 最强大的调试开关（xtrace）。它会把脚本执行的每一行命令及其扩展后的变量值都显示在屏幕上，前面通常带一个 + 号。
- **调试方法**：可以通过命令行启动时添加参数，或者直接修改脚本的第一行（Shebang）来开启这个功能。

✅ 正确答案

- シェルスクリプトを実行する際「**bash -x シェルスクリプト名**」とする
- シェルスクリプトの1行目のシバンを「**#!/bin/bash**」から「**#!/bin/bash -x**」に書き換える

🔍 选项解析与详细说明

1. bash -x 脚本名 —— 正确 ✅

- **作用**：在不修改脚本文件内容的前提下，临时开启调试模式。
- **效果**：你会看到每一行被执行的命令，非常直观。

2. 修改 Shebang 为 #!/bin/bash -x —— 正确 ✅

- **作用**：将调试模式“固化”在脚本里。

- **效果**：以后只要运行这个脚本（如 ./test.sh），它都会自动开启调试模式。

✗ 为什么其他选项不对？

- 「**echo コマンドで変数の値を出力する**」：
 - **评价**：这确实是一种调试手段，但在题目要求“**确认整个脚本的处理流程（全体の処理の流れ）**”时，手动埋入 echo 太慢且不全面。相比之下，-x 是系统自动完成的“全记录”。
- 「**bash -X（大写）**」：
 - **错误**：Bash 的调试开关是小写的 -x。大写 -X 没有这个功能。
- 「**declare -f**」：
 - **错误**：这个命令是用来列出当前 Shell 中定义的“函数内容”的，无法追踪脚本的执行流程。
- 「**declare -f**」：
 - **错误**：这个命令是用来列出当前 Shell 中定义的“函数内容”的，无法追踪脚本的执行流程。

考试小秘籍

1. 区分 -x 和 -v：

- -v 是你写了什么它显示什么（原文）。
- -x 是变量替换后的样子（实战结果）。考试问“追踪流程”首选 -x。

2. 临时 vs 永久：

- 想临时看一眼？用 `bash -x script.sh`。
- 想以后一直看？改第一行 `#!/bin/bash -x`。

3. **局部调试**：虽然题目没考，但记住在脚本内部用 `set -x` 开启、`set +x` 关闭，可以只调试其中一小段代码。

22. 我们把这张图想象成一场**“接力比赛”**。哪怕你从来没摸过电脑，只要跟着这三个角色的动作走，你就能彻底明白发生了什么。

```
$ cat ./script.sh
#!/bin/bash
exit $#
$
$ ./script.sh
$ echo $?
```

知识点总结

我们要拆解的三个核心动作是：

1. **看 (cat)**：看脚本写了什么。
2. **跑 (./)**：让脚本演出来。
3. **问 (\$?)**：打听刚才演得好不好。

第一步：cat ./script.sh —— 读剧本

- **什么是 cat?**：它就像是你的**眼睛**。你对着一个文件喊 cat，它就把里面的文字打印在屏幕上给你看。
- **剧本里写了什么?**：
 - 第一行：#!/bin/bash（这是告诉系统：我是个 Bash 脚本）。
 - 第二行：**exit \$#**（这是核心!）。
 - **\$# 是什么?**：它是一个**“计数员”**。它专门数一数：在这个脚本运行的时候，屁股后面跟了几个参数（小跟班）。

第二步：./script.sh —— 演员上场

- **动作**：图里显示执行了 ./script.sh。
- **关键点**：你看这个名字后面是**空的**，什么都没跟。
- **计数员工作**：既然名字后面什么都没有，**\$#（计数员）数出来的结果就是 0**。
- **结果**：脚本执行了 exit 0，然后就“杀青”下场了。

第三步：echo \$? —— 采访导演

- **什么是 echo?**：它就像是一个**大喇叭**，负责把信息喊出来。
- **什么是 \$?**：它是一个**“小侦探”**。它**不关心脚本里写了什么，它只关心上一个演员下场时丢下的数字是多少**。
- **侦察结果**：刚才脚本执行的是 exit 0，所以小侦探抓到的数字就是 **0**。
- **最终喇叭喊出**：**0**。

23. 新しいシェルを起動するコマンドは **bash**

在 Linux 的世界里，**bash** 不仅仅是一个环境的名字，它本身也是一个**程序（可执行文件）**。

当你直接在终端输入 **bash** 并回车时，就像你在 Windows 里又点开了一个新的 **Chrome 窗口**，或者在手机上又打开了一个新的 **App 界面**。

注意命令位置：

- `ls 2> fileA`：只把**这一条命令**的错误存起来。
- `bash 2> fileA`：把**接下来整个会话**的错误都存起来，直到你输入 `exit` 为止。

分清“谁”在重定向：题目问“为什么是新开启”，是因为 **bash** 本身就是开启新环境的指令。如果换成 `python 2> fileA`，那就是开启一个新的 Python 环境并记录它的错误。

24. シェルスクリプトにおいて、条件分岐の書式に関する以下のAとBに当てはまる語句はどれか。

if文はifで開始し、[A]で終了する。

case文はcaseで開始し、[B]で終了する。

这道题考查的是 Bash 脚本中**条件分支（条件分歧）语句**的闭合语法。Bash 的设计者非常幽默，他们规定：**很多语句的结束标志就是把开始标志的字母倒过来写。**

💡 知识点总结

在 Bash 脚本中，为了让系统知道一段逻辑在哪里结束，使用了以下规则：

if 语句：以 `if` 开始，以 `fi`（if 倒过来）结束。

case 语句：以 `case` 开始，以 `esac`（case 倒过来）结束。

为什么是倒着写？

这种“镜像写法”是 Bash 的语法特色，目的是为了清晰地界定代码块的边界：

`if` $\rightarrow$ 做判断 $\rightarrow$ `fi`（结束）

`case` $\rightarrow$ 做多项选择 $\rightarrow$ `esac`（结束）

25. 在 Bash 脚本中，为了让代码看起来更像传统的编程语言，人们给 `test` 命令发明了一个更简洁的写法。

💡 知识点总结

test 命令：用于检查某个条件是否成立（比如文件是否存在、数字谁大）。

中括号 []：它是 `test` 命令的**完全等价代换**。

强制规则：使用中括号时，括号内侧**必须保留空格**。

✅ 正确答案

[条件式]

🔍 选项解析与详细说明

1. 为什么是 []？

在 Linux 系统中，其实真的有一个程序就叫 `[`（你可以去 `/usr/bin/[` 找找看）。它的功能和 `test` 一模一样。

写法 A: `test -e file`

写法 B: `[ -e file ]` 这两行代码在系统眼里是**完全一回事**。

26. 这道题考查的是 `chmod`（**权限变更**）命令的两种表达方式：**符号模式（文字）**和**八进制模式（数字）**。

权限的三组人：文件所有者（u）、所属组（g）、其他人（o）。全部合起来叫所有人（a）。

权限的三种事：读（r=4）、写（w=2）、**执行（x=1）**。

chmod 的加减法：使用 `+` 代表增加权限，`-` 代表去掉权限。

`chmod a+x sample.sh`（给所有人加上执行权）= `chmod +111 sample.sh`（给三组人各加上 1 分的执行权）

27. 下図のシェルスクリプト「test.sh」では、カレントディレクトリのファイル名を1行に1つずつ表示している。スクリプトの空欄に当てはまるものはどれか。

(2つ選択)

ls

``ls``

all

*

.

シェルスクリプトの **for** ループにおいて、どのようにして「ファイル名のリスト」を渡すかを問うています。

```
■ スクリプト「test.sh」
#!/bin/bash
for FNAME in (    )
do
    echo $FNAME
done

■ test.shの実行結果
$ ./test.sh
file1.txt
file2.txt
file3.txt.xz
test.sh
```

for 循环的格式: `for 变量 in 列表; do 命令; done`。这里的“列表”需要是一系列以空格分隔的字符串。

通配符（ワイルドカード）: * 代表当前目录下的“所有文件”。

命令替换（コマンド置換）: 使用反引号 ``` 可以执行一个命令，并将它的输出结果作为参数传递给 `for`。

* (星号通配符) —— 正确 ✓

逻辑: 在 Shell 中, * 会被自动展开为当前目录下所有的文件名。

执行过程: `for FNAME in *` 实际上变成了 `for FNAME in file1.txt file2.txt ...`。这是最简单、最标准的方法。

``ls`` (带反引号的 ls) —— 正确 ✓

逻辑: 反引号告诉 Shell: “先执行里面的 `ls` 命令, 然后把得到的结果放在这里”。

执行过程: `ls` 列出了所有文件, 这些文件名随后被填入 `in` 后面。

✗ 为什么其他选项不对?

`ls` (不带反引号):

如果你写 `for FNAME in ls`, Shell 会认为列表里只有一个单词叫 "ls", 结果只会打印出一个单词 `ls`, 而不会去执行这个命令。

`all`:

`all` 只是一个普通单词, 并不是 Shell 的关键字, 无法代表所有文件。

`.` (点号):

在 Linux 中 `.` 代表“当前目录”本身。如果你写 `for FNAME in .`，它只会打印出一个点 `.`，而不会列出里面的文件。

方法	语法	它是如何工作的？	形象理解
通配符	<code>*</code>	Shell 看到星星，自动变身成所有文件名	魔法变身
命令替换	<code>`ls`</code>	先跑一遍 ls，把结果抄在 in 后面	借力打力

 考试小秘籍

反引号 vs 括号：在现代 Shell 中，`$(ls)` 和 ``ls`` 的效果是一样的，都是命令替换。如果选项里有 `$(ls)` 也是对的。

隐藏文件陷阱：注意，`*` 默认是**不包含**以 `.` 开头的隐藏文件的（比如 `.bashrc`）。如果要包含它们，通常需要特殊设置。

空格问题：如果文件名里有空格，使用 ``ls`` 可能会出问题（会被拆成两个词），而使用 `*` 则相对更安全一些。

`if` 结束变 `fi`，（字母倒写）

`case` 结束变 `esac`，（字母倒写）

`for/while` 循环不倒写，干完活了喊 `done`！（搞定收工）

28. 下图のシェルスクリプト「sample.sh」について、正しい実行結果は？

```
■スクリプト「sample.sh」
#!/bin/bash
shift
echo $1,$2,$3,$4,$5,$6,$7,$8,$9

■sample.shの実行結果
$ ./sample.sh a b c d e f g h i j k l
( )
```

这道题考查的是 Bash 脚本中的 `shift` 命令 以及 位置参数（位置パラメータ） 的处理。

 知识点总结

位置参数：当你执行 `./sample.sh a b c` 时，`$1` 是 `a`，`$2` 是 `b`，以此类推。

shift 命令：它的作用是把所有的参数**向左移动一位**。原来的 `$1` 会被丢弃，原来的 `$2` 变成新的 `$1`，原来的 `$3` 变成新的 `$2` ……

初始参数状态

运行命令是： `./sample.sh a b c d e f g h i j k l` 此时参数对应如下：

`$1` =a, `$2` =b, `$3` =c, `$4` =d, `$5` =e, `$6` =f, `$7` =g, `$8` =h, `$9` =i, `$10` =j ...

执行 `shift` 命令

由于脚本里只写了一个 `shift`（默认移动 1 位），参数发生了大漂移：

原本的 `$1` (a) 被扔掉了。

新的 `$1` = 原本的 `$2` (b)

新的 `$2` = 原本的 `$3` (c)

...

新的 `$9` = 原本的 `$10` (j)

执行 `echo` 命令

脚本要求打印：`echo $1,$2,$3,$4,$5,$6,$7,$8,$9` 把我们刚才漂移后的结果填进去，就得到了：`b,c,d,e,f,g,h,i,j`

 考试小秘籍

默认值：如果只写 `shift`，它等同于 `shift 1`。如果题目写 `shift 2`，那就要扔掉前两个，从 `c` 开始数。

`$0` 不受影响：注意！`$0` 永远是脚本名字（`./sample.sh`），`shift` 命令**绝对不会**去动 `$0`。

循环中的妙用：在处理不确定数量的参数时，经常用 `while` 循环配合 `shift`，处理完第一个就挪走一个，直到参数全部处理完。

29. 下图のシェルスクリプト「sample.sh」の実行結果として適切なものは次のうちどれか。ただし、このスクリプトファイルの書式について正しいとは限らないものとする。

```
■スクリプト「sample.sh」
#!/bin/bash
echo $1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11

■sample.shの実行結果
$ ./sample.sh a b c d e f g h i j k
(          )
```

这道题考查的是 Bash 脚本中 **位置参数（Positional Parameters）** 的特殊规则。你之所以觉得应该是 `$a$` 到 `$k$`，是因为直觉上认为 `$10` 会被识别为“第 10 个参数”，但在 Bash 的默认规则里，事情并没有那么简单。

 知识点总结

- **个位数参数：**`$1` 到 `$9` 可以直接使用。
- **两位数参数陷阱：**如果不加花括号，`$10` 会被拆解为 `$1` 后面跟着一个字符 `0`。
- **正确引用方式：**要引用第 10 个及以后的参数，必须使用花括号，例如 `${10}`。

 正确答案

a,b,c,d,e,f,g,h,i,a0,a1

🔍 选项解析与详细说明

为什么结果是 a0 和 a1？

观察脚本中的 echo 部分：echo \$1, \$2, ..., \$9, \$10, \$11。

- **\$1 到 \$9**：分别对应输入的 a 到 i，这部分没有争议。
- **\$10 的真相**：Bash 会先解析 \$1（即 a），然后把剩下的 0 当作普通文本拼接在后面。结果就是 a0。
- **\$11 的真相**：同理，解析 \$1（即 a），后面拼接字符 1。结果就是 a1。

为什么不是 a 到 k？

如果你想要输出 j（第 10 个参数）和 k（第 11 个参数），脚本必须写成：

```
echo $1, $2, ..., $9, ${10}, ${11}。
```

由于题目给出的脚本没有加花括号，它就无法识别两位数的索引。

Bash 的环境变量符号 \$ 是个“单眼皮”，它默认只能看清一位数字。

看到 \$10，它只认出了 \$1 是个熟人（参数），后面的 0 只是路人甲。

如果你想让它看清两位数，就得给它戴副眼镜（花括号 { }）！没戴眼镜看 \$10？那就是 \$1 后面跟个 0！

📝 考试小秘籍

- 观察是否有花括号**：看到 \$10 或 \$11 时，第一反应就是检查有没有 { }。
- 定位 \$1**：一旦确定没有花括号，立刻看 \$1 是什么，然后在它后面直接贴上数字 0 或 1。
- 注意 shift（参考另一张图）**：如果在 echo 之前执行了 shift，所有参数会整体向前移动一位（\$2\$ 变 \$1\$，\$3\$ 变 \$2\$）。但在本题图中，sample.sh 并没有 shift 命令，所以直接按原位计算。

30. 全ての引数が格納されるシェルの特殊変数は？

这道题考查的是 **Bash 特殊变量（特殊变数）** 中用于代表“所有参数”的符号。

💡 知识点总结

当你在命令行输入 ./script.sh a b c 时：

\$* 和 \$@：都代表了所有的参数（a, b, c）。

两者的区别：虽然它们都表示“全部”，但在加上双引号时（如 "\$*" vs "\$@"），处理方式会有细微差别。

1. \$* (Asterisk) —— 正确 ✓

含义：代表所有的位置参数。

特点：如果加上双引号 "\$*"，它会将所有参数看作**一个整体**（一个长字符串），中间用空格隔开。

2. \$@ (At sign) —— 正确 ✓

含义：代表所有的位置参数。

特点：如果加上双引号 "\$@"，它会将每个参数看作**独立的个体**。这在循环处理带空格的文件名时非常安全。

✗ 为什么其他选项不对？

\$#：

含义：代表参数的**个数**（是个数字）。就像我们之前题里看到的，没带参数就是 0。

\$0：

含义：代表**脚本文件本身的名字**。

\$?：

含义：代表**上一个命令的退出状态码**（0 代表成功，非 0 代表失败）。

变量	英文含义	形象比喻
\$0	Script Name	演员的名字 (脚本名)
\$1 ~ \$9	Positional Args	第 1~9 号观众
\$*	All Args	全体观众 (挤在一起)
\$@	All Args	全体观众 (每个都有座位)
\$#	Number of Args	观众的总人数
\$?	Exit Status	演完后的评价分数

📝 考试小秘籍

必考双子星：看到“全部引数（参数）”，脑子里立刻跳出**星号 *** 和 **艾特符 @**。

区分计数与内容：

问“**内容**” \$\rightarrow\$ 选 \$* / @\$。

问“**个数**” \$\rightarrow\$ 选 \$#。

注意 \$0：它不属于“参数（引数）”，它是命令本身。如果题目问“第一个参数”，选 \$1，千万别选 \$0。

31. 現在の環境変数PATHに新たなパス「/home/public」を追加するスクリプト「setpath.sh」を作成した。ファイルの内容とパーミッションは以下の通りである。

```
$ cat setpath.sh
#!/bin/bash
PATH=$PATH:/home/public
$ ls -l setpath.sh
-rw-rw-r--. 1 test test 36 Jun  8 09:03 setpath.sh
```

このスクリプトの設定を即座に反映させるコマンドは？

这道题考查的是 **Shell 环境变量的生效范围**，是 Linux 考试中非常经典的一个坑。

💡 知识点总结

父进程与子进程：普通的执行方式（如 `./` 或 `sh`）会开启一个“子 Shell”。子 Shell 里的变量修改（比如改 PATH）在它结束时就消失了，**无法影响你现在的“父 Shell”**。

Source 加载：`source` 命令（或者它的缩写 `.`）会直接在**当前 Shell** 中运行脚本里的内容。

✅ 正确答案

`source setpath.sh`

`. setpath.sh`

🔍 选项解析与详细说明

1. `source` 与 `.` —— 正确 ✅

逻辑：这两个命令是完全等价的。它们不会开启新进程，而是把脚本里的 `PATH=...` 直接在当前窗口执行。

效果：执行后，你立刻输入 `echo $PATH` 就能看到新路径已经加进去了。

❌ 为什么其他选项不对？

`./setpath.sh`：

问题：它会开启一个“子娃娃”进程。子娃娃虽然改了自己的 PATH，但一跑完它就“死”了，你所在的父窗口 PATH 没有任何变化。

额外障碍：图中显示该文件权限是 `-rw-rw-r--`，**没有执行权 (x)**，所以直接用 `./` 运行会报 `Permission denied`。

`sh setpath.sh`：

问题：同样是开启子进程，无法改变当前环境。

`exe setpath.sh`：

错误：这不是一个合法的加载脚本命令（正确的叫 `exec`，但 `exec` 会用脚本替换掉当前进程并退出，通常不用于这种场景）。

追踪过程找“小 x” (`-x`)；（像 X 光一样看透执行过程）

即时生效点一点 (`.`)，（点号或者 `source`）

子进程里改不动，自家客厅才亮灯！

32. カレントディレクトリでのlsコマンドの結果は以下である。

```
$ls
```

```
dir1 dir2 test1.txt test2.txt
```

上記と同様に、下記の実行結果にするためには、事前にどれを入力すれば良いか。（2つ選択）

```
echo RESULT
```

```
dir1 dir2 test1.txt test2.txt
```

这道题考查的是 **命令替换（コマンド置換）**。它的核心逻辑是：你想让变量 `RESULT` 存储的不是“ls”这两个字母，而是“**执行ls命令后得到的那些文件名**”。

💡 知识点总结

普通赋值： `VAR=ls` 只会让变量等于字符串 "ls"。

命令替换： 如果你想把一个命令的**执行结果**抓取并存入变量，必须给这个命令套上特殊的“外衣”。

两种外衣：

反引号： ``ls`` (Backticks)

括号组合： `$(ls)` (Dollar with Parentheses)

✅ 正确答案

```
RESULT=$(ls)
```

```
RESULT=ls``
```

🔍 选项解析与详细说明

1. `RESULT=$(ls)` 和 `RESULT=ls``` —— 正确 ✅

逻辑： 这两个符号就像是“提取器”。Shell 会先跑一遍 `ls`，把出来的 `dir1 dir2...` 抓在大印里，然后赋给 `RESULT`。

结果： `echo $RESULT` 就会打印出抓到的文件名。

❌ 为什么其他选项不对？

`RESULT="ls"` 和 `RESULT=ls`：

结果： 这两个都只是把 "ls" 这两个字母存进去了。执行 `echo $RESULT` 只会显示 `ls`，而不会显示文件列表。

`RESULT=$ls`：

结果： 这是在尝试把另一个叫 `ls` 的变量的值赋给 `RESULT`。因为之前没定义过变量 `ls`，所以结果是空的。

`RESULT=(ls)`：

结果：这在 Bash 中是定义 **数组（Array）** 的语法。虽然它能存进去，但 `echo $RESULT` 默认只会打印数组的第一个元素，且语法意图与题目不符。

`ls` 是个厨师，他的工作是做出一盘叫“文件名”的菜。如果你直接写 `RESULT=ls`，你只是记住了厨师的名字叫 `ls`。如果你写 `RESULT=$(ls)`，你是把厨师做好的那盘菜（执行结果）端到了你的桌子（变量）上！想要菜，就得加口袋（`$()` 或 ```）！

考试小秘籍

首选 `$()`：虽然反引号 ``` 也能用，但现代 Linux 更推荐使用 `$()`，因为它支持嵌套（大口袋套小口袋），且长得像引号，不容易看错。

注意空格：变量赋值时，**等号 `=` 两边绝对不能有空格！**例如 `RESULT = $(ls)` 是会报错的。

双引号的作用：如果你写 `echo "$RESULT"`（带双引号），它会保持原来的换行格式；如果不带双引号 `echo $RESULT`，所有的换行都会变成空格显示在一行（正如题目所示）。

33. バックアップをとった日付を記録する目的で、以下の環境を構築した。

```
$ ls -l
合計 4
drwxr-xr-x. 2 staff staff 6 11月 13 10:33 backup
-rwxr-xr-x. 1 staff staff 58 11月 13 10:34 backup.sh

$ cat backup.sh
#!/bin/bash

if test -e backup; then
    date > backup
fi
```

backup.shを実行した結果はどうなるか。

这道题考查的是 **if 条件判断** 与 **Linux 文件类型** 之间的相互作用。这是一个非常经典的“视觉陷阱”题。

知识点总结

test -e (Exists)：判断某个名称是否存在。不管它是文件（File）还是目录（Directory），只要名字对上了，就算存在。

重定向 `>`：将输出写入文件。如果目标是一个**目录**而非文件，这个操作会失败。

正确答案

エラーになる

选项解析与详细说明

我们来根据图片内容进行逻辑推理：

检查 **if** 条件：`if test -e backup; then`

观察 `ls -l` 的结果：第一行是 `drwxr-xr-x... backup`。

开头的 `d` 说明 `backup` 是一个 **目录 (Directory)**。

因为 `backup` 确实存在，所以 `test -e backup` 返回“真（成功）”，程序进入 `then` 后面的指令。

执行指令：`date > backup`

这一行的意思是：获取当前日期，并把它存进名为 `backup` 的**文件**里。

冲突点：在 Linux 中，你不能用 `>` 把内容直接塞进一个已经存在的**目录**里。重定向的目标必须是文件。

结果：系统会报错，提示类似 `bash: backup: Is a directory`（这是一个目录）。

如果脚本写的是 `test -f backup`（仅限文件），那么条件就会不成立，脚本会直接退出而不会报错。

题目故意用 `-e`（存在就行），就是为了诱导你进入那个会报错的 `date > backup` 环节。

34. スクリプトファイルの1行目に「#!」とバイナリファイルへのパスが記載してある。これは**インタプリタ**

💡 知识点总结

- **Shebang**：指的是脚本文件第一行的 `#!` 符号。
- **作用**：告诉内核（Kernel）该用哪一个程序来读取并执行这个脚本。
- **本质**：指定 **解释器（インタプリタ / Interpreter）**。

为什么是“解释器”？

脚本文件本质上只是文本。计算机看不懂文本，它需要一个“翻译官”。

如果第一行是 `#!/bin/bash`，翻译官就是 Bash。

如果第一行是 `#!/usr/bin/python3`，翻译官就是 Python。

这个“翻译官”在计算机专业术语里就叫 **インタプリタ (Interpreter)**。

❌ 为什么其他选项不对？

ヘッダ (Header)：虽然它在文件的最头部，但在 Linux 术语中，它特指“解释器指令”，而不是通用的数据头。

サンプルプログラム (Sample)：这只是个词汇，和语法功能无关。

コメント (Comment)：

易错点：虽然 `#` 在 Shell 中代表注释，但当 `#!` 连在一起出现在**第一行**时，它具有特殊的魔力，不再是普通的注释，而是一条指令。

環境変数 (Environment Variable)：环境变量是类似 `PATH` 这种存储在内存里的配置，不是写在文件第一行的路径。

看清符号：必须是 `#!`（井号+感叹号）连在一起才叫 Shebang。

必须在第一行：如果这一行写在第二行或者后面，它就会退化成普通的“注释”，失去调用解释器的功能。

绝对路径：`#!` 后面必须跟着解释器的**绝对路径**（从 `/` 开始），不能只写个名字。

35. 文字列1(STR1)と文字列2(STR2)が等しい、もしくは数値1(INT1)と数値2(INT2)が等しい場合のいずれかの場合に、「[OK]」と表示するシェルスクリプトを作成したい。下図の()内に記述する内容として適切なものは次のうちどれか。

```
#!/bin/bash
STR1=/home/pingt/jpgs
STR2=/home/pingt/jpgs
INT1=255
INT2=128
if test $STR1 (1) $STR2 (2) $INT1 (3)
$INT2
then
    echo "[OK]"
else
    echo "[NG]"
fi
```

这道题考查的是 **Bash 条件判断**（`test` 命令）中的三个核心语法：**字符串比较**、**逻辑连接词** 和 **数值比较**。

 知识点总结

字符串比较（文字列比較）：使用 `=`。

数值比较（数値比較）：使用 `-eq` (Equal)。

逻辑连接词（論理結合）：

`-o`：Or（或者）。只要其中一个条件成立就为真。

`-a`：And（并且）。必须两个条件都成立才为真。

 正确答案

**** (1) = (2) -o (3) -eq ****

 选项解析与详细说明

我们根据题目的要求“**字符串相等 OR 数值相等**”来填空：

(1) 字符串比较：`$STR1 (1) $STR2`

需求：判断两个路径字符串是否相同。

语法：在 `test` 命令中，字符串的“相等”用一个等号 `=`。

结果：(1) 填 `=`。

(2) 逻辑连接：条件A (2) 条件B

需求：题目说的是“もしくは（或者）”、“いずれかの場合（其中之一）”。

语法：表示“或者”的逻辑词是 `-o` (Or)。

结果：(2) 填 `-o`。

(3) 数值比较：\$INT1 (3) \$INT2

需求：判断两个数字（255 和 128）是否相等。

语法：在 `test` 中，数字比较不能用等号，必须用 `-eq`。

结果：(3) 填 `-eq`。

字母用等号 (`=`)，数字用杠杠 (`-`)。Or 是“偶” (`-o`)，或者就行；And 是“俺” (`-a`)，两个都要！

`-lt`：Little (小)。`-gt`：Great (大)。`-eq`：Equal (等)。

 考试小秘籍

一眼定生死：看到比较的是 `STR`（字符串），立刻排除所有带 `-eq` 的选项；看到比较的是 `INT`（数值），立刻排除所有带 `=` 的选项。

区分逻辑词：

题目有“もしくは”、“または” $\rightarrow$ 选 `-o`。

题目有“かつ”、“および” $\rightarrow$ 选 `-a`。

注意空格：虽然这道题是填空，但在实际写脚本时，每个符号两边都必须有空格，比如 `test $A = $B`，漏了空格就会报错。

条件式	説明
数値1 -eq 数値2	数値1と数値2が等しければ真 (= や equal の意味)
数値1 -ge 数値2	数値1が数値2以上であれば真 (>= や greater than or equal の意味)
数値1 -gt 数値2	数値1が数値2より大きければ真 (> や greater than の意味)
数値1 -le 数値2	数値1が数値2以下であれば真 (<= や less than or equal の意味)
数値1 -lt 数値2	数値1が数値2未満であれば真 (< や less than の意味)
数値1 -ne 数値2	数値1と数値2が等しくなければ真 (<> や not equal の意味)

`-eq` (Equal): 是等于, 两头相等没差距。

`-ne` (Not Equal): 不等于, 你我之间有分歧。

`-gt` (Great): 大于你, 大个 (G) 顶天更有力。

`-lt` (Little): 小于你, 小弟 (L) 低头受委屈。

`-ge` (Great Equal): 大于等, 大个 (G) 带个小尾巴 (e)。

`-le` (Little Equal): 小于等, 小弟 (L) 带个小尾巴 (e)。

条件式	説明
文字列1 = 文字列2	文字列1と文字列2が等しければ真
文字列1 != 文字列2	文字列1と文字列2が等しくなければ真

一个等号 (`=`): 你我真像, 一模一样。叹号等号 (`!=`): 你要不同, 才是真理。

看清变量类型:

- 变量名带 `STR` 或内容是路径/单词 $\rightarrow$ 用 `=` 或 `!=`。
- 变量名带 `INT` 或内容是纯数字 $\rightarrow$ 用 `-eq` 或 `-ne`。

36. 以下のシェルスクリプトを引数を与えずに実行した際の結果について

这道题考查的是 `$#` (参数个数) 特殊变量与 `!` (逻辑非) 运算符的组合应用。

💡 知识点总结

`$#`: 代表传递给脚本的参数个数。

`test !`: 代表“非”运算, 会将后面的结果取反 (真变假, 假变真)。

`-eq`: 数值比较中的“等于”。

✅ 正确答案

数值評価の結果は「0 -eq 0」により真となるが、オプション「!」が指定されているため「[NG]」が表示される

🔍 选项解析与详细说明

我们来模拟脚本的执行过程:

1. 确定 `$#` 的值

题目要求: 「引数を与えずに実行」(不带参数执行)。

此时, 参数个数为 0。所以, 变量 `$#` 的值就是 0。

2. 计算括号内的数值比较

表达式: `$# -eq 0` $\rightarrow$ 代入后变为 `0 -eq 0`。

结果: 真 (True)。

3. 应用 `!` 取反插件

表达式： `test ! (0 -eq 0)` 。

逻辑：原本是“真”，加上 `!` 后变成了“假” (False)。

4. 执行 `if` 分支

因为结果为“假”，程序跳过 `then`，进入 `else` 分支。

最终屏幕打印： `[NG]` 。

 考试小秘籍

先算右边，再算左边：看到 `!` 不要慌，先算出 `!` 后面的表达式是对是错，最后再反转一下。

死记 `$#`：只要看到 `$#`，立刻在脑子里翻译成“参数的个数”。

如果题目说执行命令是 `./test.sh a b`，那 `$#` 就是 2。

如果题目说“引数なし”，那 `$#` 永远是 0。

取反的意义：在实际脚本中，`if test ! -f file` 经常用来表达“如果文件**不存在**的话”，是一种非常实用的反向思维。

 逻辑陷阱“三步走”口诀

第一步，看报数：`$#` 报个数，没给参数就是 `0`。

第二步，算等号：`0 -eq 0` 没毛病，结论为“真”对对对。

第三步，见叹号：前面叹号 (`!`) 唱反调，对的非要说成错。

结果错，走 `else`，奖杯领个 `[NG]` 回！

 考点形象记忆法

如果口诀太长，记这三个“身份标签”：

符号	身份	对应本题动作
<code>\$#</code>	报数员	发现没参数，大喊：“0！”
<code>-eq 0</code>	裁判员	看到 0 等于 0，判定：“正确！”
<code>!</code>	杠精	只要是正确的，他非要跳出来：“不对！”

 考试小秘籍

引数 (参数) 计数：

题目说「引数なし」：`$#` = 0

题目说「引数 a b」：`$#` = 2

! 的魔力：看到 `!` 出现在 `test` 后面，脑子里立刻准备“翻转”结果。

分支选择：

结论为 真 $\rightarrow$ 执行 `then` (通常是 OK)

结论为 假 $\rightarrow$ 执行 `else` (通常是 NG)

37. 这道题考查的是 **文件时间戳 (タイムスタンプ) 的比较**。在 Linux 脚本中，除了比较数字和字符串，我们经常需要比较两个文件的“新旧”。

💡 知识点总结

- `-nt` (**Newer Than**): 判断左边的文件是否比右边的文件**新** (修改时间更晚)。
- `-ot` (**Older Than**): 判断左边的文件是否比右边的文件**旧** (修改时间更早)。

✅ 正确答案

`-ot`

🔍 选项解析与详细说明

我们来观察题目中的“前提条件” (`ls -l` 的结果)：

- a. `/home/file1`：修改时间是 **Jan 11 18:52**。
- b. `/home/file2`：修改时间是 **Jan 11 22:48**。

逻辑推理：

- 18:52 早于 22:48。
- 所以，`file1` 比 `file2` **更旧 (更早)**。
- 题目要求显示 `[OK]`，也就是说 `if` 的条件必须成立。
- 既然 `file1` 确实比 `file2` 旧，我们就需要填入表示“旧”的符号：`-ot`。

📝 考试小秘籍

- a. **看清时间**：考试时不仅要看日期，还要看具体的**小时和分钟**。
- b. **对应关系**：

- 左边时间 < 右边时间 $\rightarrow$ 使用 `-ot`。
- 左边时间 > 右边时间 $\rightarrow$ 使用 `-nt`。

c. 排除干扰：

- `-lt`、`-gt`、`-ne` 是用来比**数字**的，不能用来比文件时间。看到文件路径作为参数，优先考虑 `-nt` 或 `-ot`。

条件式	説明
ファイル1 -nt ファイル2	ファイル1の更新日付がファイル2より新しければ真 (newer than の意味)
ファイル1 -ot ファイル2	ファイル1の更新日付がファイル2より古ければ真 (older than の意味)

`-nt` (**Newer**): 新 (N) 衣服，左边更新才算数。 `-ot` (**Older**): 旧 (O) 玩具，左边更老才成立。

为什么是 `-nt` ?

我们来看题目给出的 `ls -l` 前提条件：

- a. `/home/file1`：修改时间是 **23:52**。
- b. `/home/file2`：修改时间是 **19:21**。

逻辑拆解：

- **比时间**：深夜的 23:52 肯定比傍晚的 19:21 要**晚**出现。
- **定属性**：因为 `file1` 是在更晚的时间修改的，所以 `file1` 是**“新文件”，`file2` 是“旧文件”**。
- **配对符号**：题目要求 `if` 判断成功（显示 `[OK]`），而左边的 `file1` 确实比右边的 `file2` **新 (Newer)**，所以必须填入 `-nt`。

 考试小秘籍

- c. **左侧为王**：永远判断“左边的文件相对于右边是什么状态”。左边新就选 `n`，左边旧就选 `o`。
- d. **数字越大越 New**：23 点 > 19 点，所以 23 点的文件就是 `-nt`。
- e. **避开数字干扰**：不要看到 `-ge` (大于等于) 就想去比数字大小，那是比**纯数字**用的。看到文件路径，只能在 `-nt` 和 `-ot` 之间二选一。

38. 文件测试运算符（ファイル演算子）

条件式	説明
<code>-d</code> パス	指定したパスが存在し、ディレクトリであれば真
<code>-e</code> パス	指定したパスが存在すれば真
<code>-f</code> パス	指定したパスが存在し、ファイルであれば真
<code>-r</code> パス	指定したパスが存在し、読み込み可能であれば真
<code>-w</code> パス	指定したパスが存在し、書き込み可能であれば真
<code>-x</code> パス	指定したパスが存在し、実行可能であれば真

`-d` (**D**irectory)：是目录，大门敞开 (D) 找路径。

`-e` (**E**xist)：是否存在，名字在 (E) 场就算数。

`-f` (**F**ile)：是文件，普通文件 (F) 莫认错。

`-r` (**R**ead)：可读取，读 (R) 取权限已开启。

`-w` (**W**rite)：可写入，写 (W) 入修改没问题。

`-x` (**eX**ecute)：可执行，交叉 (X) 运行跑起来。

- `-e` 与 `-f` 的区别：这是最常见的陷阱。`-e` 只要名字存在就行（包括目录），而 `-f` 必须是普通文件。
- **权限三兄弟**：`r`、`w`、`x` 对应 Linux 权限里的 `read`、`write`、`execute`，顺序和逻辑完全一致。

- **前提条件：**所有的权限检查（`rwX`）都隐含了一个前提——文件必须**先存在**。如果文件不存在，这些判断都会返回“假”。

以下の前提条件でシェルスクリプトを実行した際、「[OK]」と表示させたい。()内に記述する内容として適切なものは次のうちどれか。

```
■前提条件
# ls -l /var/log
drwx----- 2 root root 4096 Jan 11 20:06 httpd
-rw----- 1 root root 41197 Dec 20 12:34 cron

■シェルスクリプト
#!/bin/bash
if test ( ) /var/log/httpd
then
    echo "[OK]"
else
    echo "[NG]"
fi
```

💡 知识点总结

- `-d` (Directory)：判断指定路径是否为 **目录（ディレクトリ）**。
- `-f` (File)：判断指定路径是否为 **普通文件（ファイル）**。
- **数值比较符号：**`-eq` 和 `-ne` 是用于**数值**比较的，不能用于测试文件属性。

✅ 正确答案

`-d`

🔍 选项解析与详细说明

我们来观察题目中的“前提条件（前提条件）”：

a. 观察 `ls -l /var/log` 的输出：

- 第一行显示：`drwx----- 2 root root 4096 Jan 11 20:06 httpd`。

b. 识别类型：

- 权限位的第一个字母是 `d`，这明确表示 `httpd` 是一个 **目录（Directory）**。

c. 匹配脚本需求：

- 脚本内容为 `if test ( ) /var/log/httpd`。
- 题目要求执行后显示 `[OK]`，意味着 `if` 条件必须成立（为“真”）。
- 既然 `/var/log/httpd` 是个目录，我们必须填入测试目录的符号 `-d`。

❌ 为什么其他选项不对？

- `-f`：它是用来测试“普通文件”的。因为 `httpd` 是目录，填入 `-f` 会判断为假，结果会显示 `[NG]`。

- `-eq` / `-ne`：这些是数值等于或不等于的判断符号，放在这里会导致语法错误或逻辑不通。

考试小秘籍

a. **第一眼看权限位**：做这种题永远先看 `ls -l` 输出结果的第一位字符。

b. **区分 d 和 f**：

- 看到 `drwxr...` $\rightarrow$ 选 `-d`。
- 看到 `-rwxr...` $\rightarrow$ 选 `-f`。

c. **排除法**：凡是路径比较，直接划掉所有带 `eq`、`ne`、`gt`、`lt` 的数字比较选项。

39. 定義されているシェル関数を確認するためのコマンドは、「**declare -f**」

`declare` 是 Bash 的内置命令，用来声明变量或函数。

- **-f (function)**：专门指向函数。当你输入 `declare -f` 时，Bash 会把内存里存着的所有函数代码都打印出来。

首字母关联：记住 **F** 指代 **F**unction。

区分大小写：虽然 LPI 考试通常考 `declare -f`，但在实际操作中，小写 `-f` 会显示函数体代码，大写 `-F` 只显示名字。

别名混淆：题目经常把 `alias` 和 `declare -f` 放在一起，记住“别名”和“函数”在 Linux 里是两个不同的篮子。

40. bashコマンドで、シェルが実行した行を、単純なコマンドや変数を展開して標準エラー出力に出力するオプションは-x

bashコマンドで、シェルが読み込んだこれから実行する行を標準エラー出力に出力するオプションは-v

書式: bash [オプション] シェルスクリプト	
オプション	説明
-v	シェルが読み込んだ、これから実行する行を標準エラー出力に出力する (変数があれば変数名を表示)
-x	シェルが実行した行を、単純なコマンドや変数を展開して標準エラー出力に出力する

41. 以下の実行結果を元に答えよ

```
echo HOSTNAME
```

```
testserver.localdomain
```

第一步：执行括号内的命令

- 内部命令是： `echo $HOSTNAME`。
- 根据前提条件， `$HOSTNAME` 的值是 `testserver.localdomain`。

- 所以，内部命令执行后的输出结果是字符串：`testserver.localdomain`。

第二步：进行命令替换

此时，外层的命令由 `echo $(echo $HOSTNAME)` 变成了 `echo testserver.localdomain`。

第三步：执行外层命令

最终执行 `echo testserver.localdomain`，屏幕上打印出该字符串。

把 `$( )` 想象成一个“自动拆包机”。

1. 里面那个包裹是 `echo $HOSTNAME`，拆开后发现里面是“地址字符串”。
2. `$( )` 把这个“地址”直接交给了外面的 `echo` 搬运工。
3. 搬运工直接把这个地址摆在你面前。所以不管你套了几层 `echo`，只要中间是执行命令，最后出来的还是那个值！

考试小秘籍

- a. 从里向外拆：看到嵌套的 `$( )`，先算出最里面的命令输出了什么，再把结果代入外面。
- b. 别被 `$( )` 吓到：在 Bash 中，`$(command)` 和反引号 ``command`` 的作用是一模一样的，都是命令替换。
- c. 变量 vs 字符串：
 - 如果命令是 `echo '$HOSTNAME'`（单引号），结果就是字符串 `$HOSTNAME`。
 - 只要没有特殊引号阻碍，变量永远会被解析成它的真实内容。

42. 直前に実行されたコマンドの終了ステータスが格納される特殊変数は\$?

43. 脚本执行与权限（実行と権限）

```
■ サンプルスクリプト (hoge.sh)
$ cat hoge.sh
echo "Hello Ping-t Users."

■ ( 1 ) が与えられていない場合の実行例
$ /home/test/hoge.sh
-bash: /home/test/hoge.sh: Permission denied

■ ( 2 ) コマンド使用例
$ ( 2 ) /home/test/hoge.sh
Hello Ping-t Users.
```

(実行権) が無いシェルスクリプトを実行するには、(source) コマンドを使用します。ただし、(source) コマンドを実行するには、シェルスクリプトに (読み取り権) と、シェルスクリプトが存在するディレクトリに (実行権) (厳密には検索権) が必要です。なお、(source) コマンドは「. (ドット)」で代用できます。

(1) 为什么是“执行权”？

看图片中段的报错：`./hoge.sh: Permission denied`。在 Linux 中，如果你想通过路径直接运行一个文件，你必须给它“跑起来”的许可。哪怕你是个管理员，如果没有 **执行权 (Execute Permission)**，系统也会把你拦在门外。

(2) 为什么是“source”？

看图片下段的执行：`( 2 ) /home/test/hoge.sh` 成功输出了结果。结合题目描述“该命令可以用 `.` 代替”，这指的就是 **source** 命令。它是一种“绕过”执行权限限制的特种方法。

(3) 为什么是“读取权”？

source 的本质是：“把脚本里的文字读出来，在当前的屏幕里再打一遍。”既然要“读出来”，那文件就必须具备 **读取权 (Read Permission)**。如果连看都不让看，**source** 也没法替你运行。

看到 **Permission denied**：如果题目是直接运行脚本，那空缺的一定是 **执行权**。

看到 **.** (点号)：立刻联想 **source**，并且记住它只需要 **读取权**。

目录权限 (冷知识)：题目中提到了“检索权 (搜索权)”，在 Linux 目录里，进入一个目录需要 **执行权 (x)**，这被称为搜索权。虽然本题没考，但在更高阶的题目里经常出现。

44. Bash 中 定义函数 (関数定義 / Function Definition) 的语法关键字

```
# [ ] lsLR() { ls -lR $1; }
```

💡 知识点总结

- **function**：Bash 的内置关键字，用于明确声明一个函数的开始。
- **函数语法**：定义函数的标准格式有两种：
 - `function 函数名 { 命令; }` (如本题所示)
 - `函数名() { 命令; }`
- **注意点**：虽然在某些 Shell 中 **function** 关键字可以省略，但题目明确要求使用“内置命令”来填空，且格式符合 **function** 的用法。

🔍 选项解析与详细说明

我们来看这行命令的结构：`# [ function ] lsLR() { ls -lR $1; }`

- 引导词**：**function** 告诉 Shell，后面跟着的 **lsLR** 不是一个普通变量或文件，而是一段可以重复调用的代码块。
- 函数名与括号**：**lsLR()** 是函数的名字。
- 大括号 { }**：包裹着具体的执行内容，即 `ls -lR $1` (列出指定目录下的所有文件，包括子目录)。

❌ 为什么其他选项不对？

- **alias**：这是用来创建“别名”的（如 `alias ll='ls -l'`）。别名后面不能直接带 `()` 和 `{ }` 这种复杂的逻辑结构。
- **source**：这是用来“执行”一个已经写好的脚本文件的，不是用来“定义”函数的。
- **while**：这是循环语句的开头（只要...就...），完全不是定义函数的逻辑。
- **sh**：这是调用 Shell 解释器的命令。

考试小秘籍

- 固定搭配**：看到 `名字() { ... }` 这种结构，前面的空位 `$99\%$` 填 **function**。
- 分号必记**：在大括号 `{ }` 里的最后一条命令后面，一定要加 **分号 ;**，否则会报错（图中已经给出了 `;`）。
- 参数占位符**：函数里的 `$1` 代表你调用函数时传进去的第一个词。